CHAPTER 1

DESIGN A URL SHORTENING SERVICE/TINYURL

1 System Design: TinyURL 2

Decode the System Design of a URL shortening service like TinyURL, emphasizing requirements like encoding, scalability, and high readability.

1 System Design: TinyURL

1.1 Introduction

URL(Uniform Resource Locator (URL), is an address that uniquely identifies a resource on the web.) **shortening** is a service that produces short aliases for long URLs, commonly referred to as **short links**. Upon clicking, these short links direct to the original URLs. The following illustration depicts how the process works:

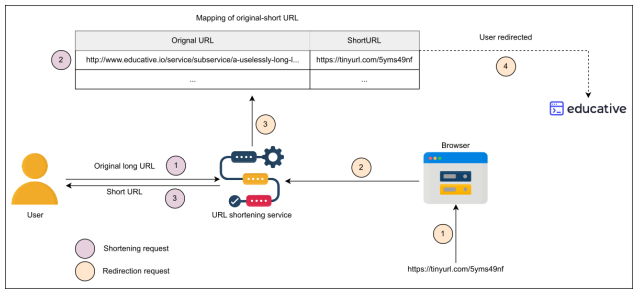


Figure 1.1: How URL shortening service works

Advantages

The key advantages of a URL shortening service are:

- Shortened URLs are convenient to use: they optimize link usage across different forms of devices due to their enhanced accessibility and non-breakability.
- They are visually professional, engaging and facilitate a higher degree of sharing possibilities.
- They are less error-prone while typing.
- They require smaller storage space at the user's end.

Disadvantages

The URL shortening service has some associated drawbacks as well. Some of them are as follows:

- We lose the originality of our brand by using a short URL generated by a third-party service. Many different brands using the same service get short URLs containing the same domain.
- Since we're using a third-party service for URL shortening, the possibility of it getting shut down and wiping all our shortened URLs will always be there.
- Our business brand image depends on the trustworthiness of a URL shortening service. The wrong business move might negatively impact our business. The competition for acquiring the in-demand custom short URLs is immense, so it might be possible that the best custom URLs are already taken by the time we start generating short URLs.

Several building blocks can be considered for the system design of TinyURL. Recognize these building blocks based on the following requisite functionalities in the design and provide your answer in the AI widget given below:

- It's necessary to:
  - Store the shortened URLs.

- Provide unique IDs for each URL.
- Store frequent URL-related requests to serve them efficiently.
- Limit the number of incoming requests.

Question:
List four building blocks required for designing TinyURL service
Reference Answer:
The four key building blocks are: a) Database is used to store mapping long URLs and the corresponding short URLs. b) Sequencer provides unique IDs that serve as a starting point for each short URL generation. c) Cache is used to store the most frequent short URL-related requests. d) Rate limiters are used to avoid system exploitation and put a limit on the number of incoming requests.

To move from concept to creation, we must first lay a solid foundation by specifying the requirements of the service.

1.2 Requirements for URL Shortening Design

Let's look at the functional and non-functional requirements for the service we'll be designing:

Functional requirements

- **Short URL generation**: Our service should be able to generate a unique shorter alias of the given URL.
- **Redirection**: Given a short link, our system should be able to redirect the user to the original URL.
- **Custom short links:** Users should be able to generate custom short links for their URLs using our system.
- **Deletion**: Users should be able to delete a short link generated by our system, given the rights.
- **Update**: Users should be able to update the long URL associated with the short link, given the proper rights.
- **Expiry time**: There must be a default expiration time for the short links, but users should be able to set the expiration time based on their requirements.

Note: Even though our system doesn't reuse expired short URLs, we still delete them after their five-year lifespan. Keeping them forever would cause the data store's search index to grow endlessly, increasing query time and adding unwanted latency.

Non-functional requirements

- **Availability**: Our system should be highly available, because even a fraction of a second of downtime would result in URL redirection failures. Since our system's domain is in URLs, we don't have the leverage of downtime, and our design must have fault-tolerance conditions instilled in it.
- **Scalability**: Our system should be horizontally scalable with increasing demand.
- **Readability**: The short links generated by our system should be easily readable, distinguishable, and typeable.
- **Latency**: The system should perform at low latency to provide the user with a smooth experience.
- **Unpredictability**: From a security standpoint, the short links generated by our system should be highly unpredictable. This ensures that the next-in-line short URL is not serially produced, eliminating the possibility of someone guessing all the short URLs that our system has ever produced or will produce.

Note: Predictable short URLs can be a security risk; attackers might spot patterns and guess private links. To prevent this, our system avoids sequential IDs. Instead, it generates random, unpredictable identifiers, ensuring each short URL is unique, secure, and impossible to guess.

With our requirements clearly defined, we can now quantify the scale of the system.

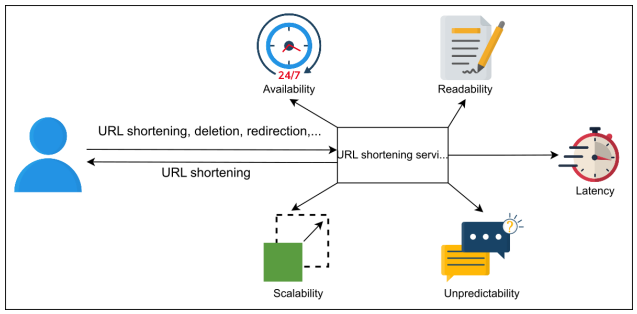


Figure 1.2: Functional and non-functional requirements

1.3 Resource estimation

It's better to have realistic estimations at the start. For instance, we might need to change them in the future based on the design modifications. Let 's make some assumptions to complete our estimation.
Assumptions

- We assume that the shortening: redirection request ratio is 1:100.
- There are 200 million new URL shortening requests per month.
- A URL shortening entry requires 500 Bytes of database storage.
- Each entry will have a maximum of five years of expiry time, unless explicitly deleted.
- There are 100 million Daily Active Users (DAU).

Storage estimation

Since entries are saved for a time period of 5 years and there are a total of 200 million entries per month, the total entries will be approximately 12 billion.
200 Million/month $\times$ 12 months/year $\times$ 5 years = 12 Billion URL shortening requests
Since each entry is 500 Bytes, the total storage estimate would be 6 TB:12 Billion $\times$ 500 Bytes = 6 TB

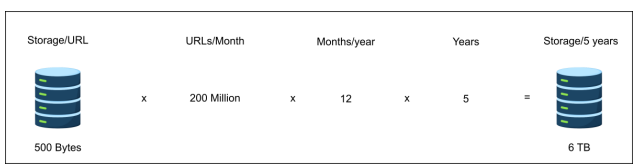


Figure 1.3: Total storage required by the URL shortening service in 5 years

Query rate estimation

Based on the estimations above, we can expect 20 billion redirection requests per month.
200 Million $\times$ 100 = 20 Billion
We can extend our calculations for Queries Per Second (QPS) for our system from this baseline. The number of seconds in one month, given the average number of days per month is 30.42:
30.42 days $\times$ 24 hours $\times$ 60 minutes $\times$ 60 seconds = 2628288 seconds
Considering the calculation above, new URL shortening requests per second will be:
$\frac{200 \text{ Million}}{2628288 \text{ seconds}} = 76$ URLs/s
With a 1 : 100 shortening to redirecting ratio, the URL redirection rate per second will be:
100 $\times$ 76 URLs/s = 7.6 K URLs/s

Bandwidth estimation

Shortening requests: The expected arrival rate will be 76 new URLs per second. The total incoming data would be 304 $Kbps$ per second:
76×500 Bytes $\times$ 8 bits = 304 Kbps
Redirection requests: Since the expected rate would be 7.6K URLs redirections per second, the total outgoing data would be 30.4$Mbps$ per second: 7.6 K $\times$ 500 Bytes $\times$ 8 bits = 30.4 Mbps

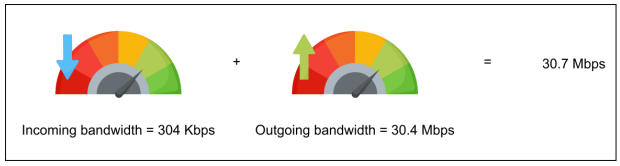


Figure 1.4: The total bandwidth required by the URL shortening service

Memory estimation

We need memory estimates in case we want to cache some of the frequently accessed URL redirection requests. Let 's assume a split of 80-20 in the incoming requests. 20 percent of redirection requests generate 80 percent of the traffic.
Since the redirection requests per second are 7.6 K, the total would be 0.66 billion for one day.
7.6 K $\times$ 3600 sec $\times$ 24 hrs = 0.66 billion
Since we would only consider caching 20 percent of these per-day redirection requests, the total memory requirements estimate would be 66 GB.
0.2 $\times$ 0.66 Billion $\times$ 500 Bytes = 66 GB

Number of servers estimation

We adopt the same approximation discussed in the Back-of-the-Envelope Calculations to calculate the number of servers needed: the number of daily active users and the requests handling limit of a server are the two main factors in depicting the total number of servers required. Recall that a typical server can serve 64,000 requests per second (RPS). Considering our assumption of using daily active users as a proxy for the number of requests per second for peak load times, we get 100 million requests per second. Then , we use the following formula to calculate the number of servers:

$$\text{Number of Servers} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

$$\text{Servers needed at peak load} = \frac{\text{100 million requests/sec}}{\text{64,000 requests/sec per server}} \approx 1562.5 \approx 1.6 \text{ K servers}$$

Figure 1.5: The number of servers required for the URL shortening service

Summarizing estimation

Based on the assumption above, the following table summarizes our estimations:

These estimations give us a tangible sense of the system's size. The next logical step is to identify the building blocks that can handle this scale.

Type of operation	Time estimates
New URLs	76/s
URL redirections	7.6 K/s
Incoming data	304 Kbps
Outgoing data	30.4 Mbps
Storage for 5 years	6 TB
Memory for cache	66 GB
Servers	1600

1.4 Building blocks we will use

With the estimations done, we can identify the key building blocks in our design. Such a list is given below:

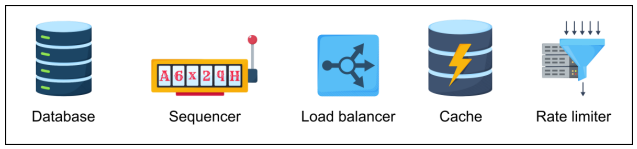


Figure 1.6: Building blocks in high level design

- **Database(s)** will be needed to store the mapping of long URLs and the corresponding short URLs.
- **Sequencer** will provide unique IDs that will serve as a starting point for each short URL generation.
- **Load balancers** at various layers will ensure smooth request distribution among available servers.
- **Caches** will be utilized to store the most frequent short URL-related requests.
- **Rate limiters** will be used to avoid system exploitation.

Besides these building blocks, we'll also need the following additional components to achieve the desired service:

- **Servers** to handle and navigate the service requests, along with running the application logic.
- A **Base-58 encoder** to transform the sequencer's numeric output to a more readable and usable alphanumeric form.

Figure 1.7: Components in high level design

With the core components identified, we need to define how clients will interact with our service. This is accomplished by designing the system's APIs.

1.5 System APIs

To expose the functionality of our service, we can use REST APIs for the following features:

- Shortening a URL
- Redirecting a short URL
- Deleting a short URL

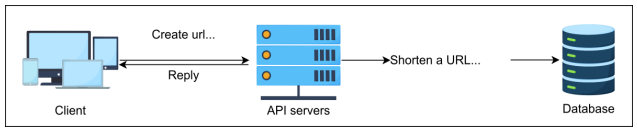


Figure 1.8: System API design overview

Shortening a URL

We can create new short URLs with the following definition:

```
shortURL(api_dev_key, original_url, custom_alias=None, expiry_date=None)
```

The API call above has the following parameters:

Parameter	Description
`api_dev_key`	A registered user account's unique identifier. This is useful in tracking a user's activity and allows the system to control the associated services accordingly.
`original_url`	The original long URL that is needed to be shortened.
`custom_alias`	The optional key that the user defines as a customer short URL.
`expiry_date`	The optional expiration date for the shortened URL.

A successful insertion returns the shortened URL. Otherwise, the system returns an appropriate error code to the user.

Redirecting a short URL

To redirect a short URL, the REST API's definition will be:

```
redirectURL(api_dev_key, url_key)
```

With the following parameters:

Parameter	Description
`api_dev_key`	The registered user account's unique identifier.
`url_key`	The shortened URL against which we need to fetch the long URL from the database.

A successful redirection lands the user to the original URL associated with the url_key.

Deleting a short URL

Similarly, to delete a short URL, the REST API's definition will be:

```
deleteURL(api_dev_key, url_key)
```

and the associated parameters will be:

A successful deletion returns a system message, URL Removed, conveying the successful URL removal from the system.

The APIs tell us what the system does. The design section, below, explains how it does it, detailing the roles of the database, the URL generator, and other key components.

1.6 Design

Let's discuss the main design components required for our URL shortening service. Our design depends on each part's functionality and progressively combines them to achieve the different workflows mentioned in the functional requirements.

Parameter	Description
`api_dev_key`	The registered user account's unique identifier.
`url_key`	The shortened URL that should be deleted from the system.

Components

We'll explain the inner mechanism of different components within our system, as well as their usage as a part of the whole system, below. We 'll also highlight the design choices made for each component to achieve the overall functionality.

Database: For services like URL shortening, there isn't a lot of data to store. However, the storage has to be horizontally scalable. The type of data we need to store includes:

- User details.
- Mappings of the URLs, that is, the long URLs that are mapped onto short URLs.

  Note: For simplicity, let's assume that our service doesn't require user registration to generate a short URL, so we can skip adding certain data to our database.

Additionally, the stored records will have no relationships among themselves other than linking the URL-creating user's details, so we don't need structured storage for record-keeping. Considering the reasons above and the fact that our system will be read-heavy, NoSQL is a suitable choice for storing data. In particular, MongoDB is a good choice for the following reasons:

1. It uses a leader-follower protocol, making it possible to use replicas for heavy reading.
2. MongoDB ensures atomicity in concurrent write operations and avoids collisions by returning duplicate-key errors for record-duplication issues.

You're evaluating different NoSQL databases for a read-intensive service. Some options on the table are Cassandra, Riak, and MongoDB. Why might Cassandra or Riak not be good choices compared to MongoDB?

Question:

Why MongoDB outperforms Cassandra and Riak?

Reference Answer:

Cassandra and Riak are leaderless and optimized for high write throughput, which makes their reads slower due to read-repair and eventual consistency. Mongo DB's leader-based model offers faster, more consistent reads—making it better suited for read-intensive workloads.

Short URL generator: Our short URL generator will comprise a building block and an additional component:

- A sequencer to generate unique IDs
- A Base-58 encoder to enhance the readability of the short URL

We built a sequencer in our building blocks section to generate 64-bit unique *numeric* IDs. However, our proposed design requires 64-bit *alphanumeric* short URLs in base-58. To convert the numeric (base-10) IDs to alphanumeric (base-58), we'll need a base-10 for the base-58 encoder. We 'll explore the rationale behind these decisions alongside the internal working of the base-58 encoder in the upcoming sections.

Take a look at the diagram below to understand how the overall short URL generation unit will work.

Other building blocks: Beside the elements mentioned above, we'll also incorporate other building blocks like load balancers, cache, and rate limiters.

- **Load balancing**: We can employ Global Server Load Balancing (GSLB) apart from local load balancing to improve availability. Since we have plenty of time between a short URL being generated and subsequently accessed, we can safely assume that our DB is geographically consistent and that distributing requests globally won't cause any issues.

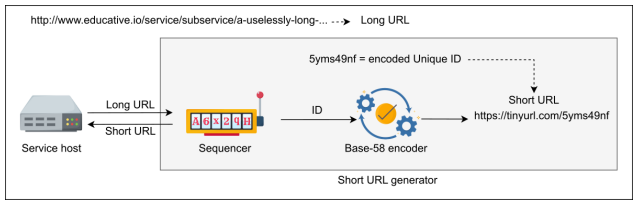


Figure 1.9: Internal working of a short URL generator

- **Cache**: For our specific read-intensive design problem, Memcached is the best choice for a cache solution. We require a simple, horizontally scalable cache system with minimal data structure requirements. Moreover, we'll have a data-center-specific caching layer to handle native requests. Having a global caching layer will result in higher latency.

- **Rate limiter**: Limiting each user's quota is preferable for adding a security layer to our system. We can achieve this by uniquely identifying users through their unique api_dev_key and applying one of the discussed rate-limiting algorithms (see Rate Limiter from *Building Blocks*). Keeping in view the simplicity of our system and the requirements, the fixed window counter algorithm would serve the purpose, as we can assign a set number of shortening and redirection operations per api_dev_key for a specific timeframe.

Quiz: Question 1: How will we maintain a unique mapping if redirection requests can go to different data centers that are geographically apart? Does our design assume that our DB is consistent geographically? **Answer:** We initially assumed that our data center was globally consistent.

Let's look at the problem differently and consider the opposite case: we need to filter the redirection requests based on data centers. Solution: An simple way of achieving this functionality is to introduce a unique character in the short URL. This special character will act as an indicator for the exact data center. Example:

Let's assume that the short URL that needs redirection is service.com/x/short123/, where 'x' indicates the data center containing this record. In this solution, if the short URL goes to the wrong data center, it can be redirected to the correct one. However, if a specific data center is not reachable for a specific short URL (and that URL is not yet cached), the redirection will fail. **Question 2:** How will the data-center-specific caching handle an unseen redirection request? **Answer:** Since we've assumed the data-center-specific caching solution for our system, a case needs highlighting: handling unseen redirection requests by our system. Scenario: The scenario entails receiving an unknown redirection request at a data center. Since the local cache wouldn't have that entry, it would fetch that record from the globally consistent database and place this entry into the local cache for future use. **Question 3:** What is the probability of collision when we ask the short URL generator for a new short URL? **Answer:** We ask the sequencer for a unique ID and by the definition of our sequencer's design, there will never be duplication in

IDs. We then encode those IDs, which also ensures no duplication. Hence, the regular short URL generation process ensures no duplication in records. Now let's consider the case of custom short URLs. Since the user is providing the short URL, there can be a duplication. We can easily calculate the probability of this collision by taking into account the size of the database containing short URL records. Let's assume there are 'n' already generated short URLs in the database. The probability that the user-provided custom short URL will be similar to an already existing one can be given by: Collision probability $= \frac{n}{\text{Total number of combinations of URL}}$ With growing 'n', the collision probability would increase, ranging from 0 when 'n=0' and 1 when 'n=total number of combinations'.

This calculation assumes that the user selects a custom short URL randomly and equally likely from the set of allowed combinations. In reality, few words are more popular than others. So,

our probability can be seen as a lower bound on the probability of collision.

Design diagram

A simple design diagram of the URL shortening system is given below.

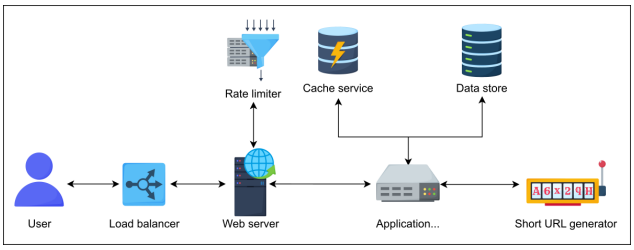


Figure 1.10: A design diagram of the URL shortening service

Workflow

Let's analyze the system in-depth and how the individual pieces fit together to provide the overall functionality.
Keeping in view the functional requirements, the workflow of the abstract design above would be as follows.

1. **Shortening**: Each new request for short link computation gets forwarded to the short URL generator (SUG) by the application server. Upon successful generation of the short link, the system sends one copy back to the user and stores the record in the database for future use.

   Quiz: Question 1: How does our system avoid duplicate short URL generation? **Answer:** * Computing a short URL for an already existing long URL is redundant,

   and the system sends the long URL to the database server to check its existence in the system. The system will check the respective entry in the cache first and then query the database. * If the short URL for the corresponding long URL is already present, the database returns the saved short URL to the application server which reroutes the requested short URL to the user. * If no short URL exists for the requested long URL, the application server asks the Short URL Generator (SUG) to compute the short URL. Once computed, the SUG sends a copy of the short URL back to the application and database servers. **Question 2:** How do we ensure that two concurrent requests for a short URL do not overwrite? **Answer:** Concurrency in handling URL shortening requests is essential, and we achieve it as follows: 1. MongoDB ensures consistency by locking and concurrency control protocols, preventing the users from modifying the same data simultaneously. 2. In MongoDB, all the write requests go through the single leader and hence exclude the possibility of race conditions due to the serialization of requests via a single leader.

2. **Redirection**: Application servers, upon receiving the redirection requests, check the storage units (caching system and database) for the required record. If found, the application server redirects the user to the associated long URL.

   Quiz: Question 1: How does our system ensure that our data store will not be a bottleneck? **Answer:** We can ensure that our data store doesn't become a bottleneck, using the following two approaches: 1. We use a range-based sequencer in our design, which ensures basic level mapping between the servers and the short URLs. We can redirect the request to the respective database for a quick search. 2. As discussed above, we can also have unique IDs for various data stores and integrate them into short URLs. We can subsequently redirect requests to the respective data store for efficient request handling. Both of these approaches ensure smooth traffic handling and mitigate the risk of the data store becoming a bottleneck.

3. **Deletion**: A logged-in user can delete a record by requesting the application server which forwards the user details and the associated URL's information to the database server for deletion. A system-initiated deletion can also be triggered upon an expiry time, as we'll see ahead.

4. **Custom short links**: This task begins with checking the eligibility of the requested short URL. The maximum length allowed is 11 alphanumeric digits. We can find the details on the allowed format and the specific digits in the upcoming lesson. Once verified, the system checks its availability in the database. If the requested URL is available, the user receives a successful short URL generation message, or an error message in the opposite case.

Managing unique ID allocation

After a custom short URL is successfully allocated, the system updates its records to maintain data integrity and prevent duplicate mappings. Each short URL is the base-58 encoding of a unique base-10 ID, which must be marked as unavailable once used.
On the backend, the server retrieves the base-10 equivalent of the generated short URL and marks it as unavailable in the ID range. This ensures that the same ID cannot be reassigned to another request. The diagram below shows how the system decodes a short URL and updates the database to track used and unused IDs.

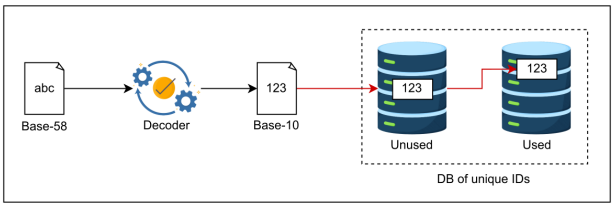


Figure 1.11: Mapping a short URL to its ID and marking it as used to prevent reuse

This mechanism also enhances availability. Instead of maintaining used and unused ID lists in memory, the system stores them in a NoSQL database, enabling faster lookups and improved scalability.
The process works as follows:
Newly generated IDs are added to the unused list.
Once an ID is assigned, it is moved to the used list.
Since base-58 encoding guarantees a one-to-one mapping with base-10 IDs, no two long URLs can share the same short URL.

Test your knowledge!

Question 1: How can we maintain a unique mapping for short URLs when redirection requests are handled by geographically distributed data centers that may not be globally consistent?

- Use a global consistent database shared by all data centers to ensure every short URL is unique and accessible from anywhere.
- **[CORRECT]** Introduce a unique character in each short URL to indicate the data center that owns it, allowing redirection to the correct location.

  Explanation: By embedding a unique character in the short URL (e.g., service.com/x/short123/), where x identifies the data center, we can route or redirect the request to the correct data center even when global consistency isn't guaranteed.
- Use a random hashing mechanism to assign short URLs to data centers dynamically at runtime.
- Replicate all short URLs across all data centers to avoid dependency on any single location.

Question 2: In a data-center-specific caching setup, how should the system handle an unseen redirection request (a short URL not found in the local cache)?

- Return an error response to the user indicating that the short URL doesn't exist.
- Forward the request to all other data centers to locate the correct cache entry.
- **[CORRECT]** Fetch the record from the globally consistent database and store it in the local cache for future requests.

  Explanation: When an unseen redirection request arrives at a data center, the system retrieves the record from the globally consistent database and adds it to the local cache. This ensures faster access for subsequent requests without depending on other data centers.
- Wait until the cache is refreshed during the next synchronization cycle.

Question 3: What is the probability of collision when generating a new short URL, and how does it differ between system-generated and user-provided custom URLs?

- For both system-generated and custom URLs, the probability of collision is the same since all IDs come from the same space.
- System-generated short URLs can collide if multiple sequencers are active, while custom URLs never collide.

  Explanation: System-generated short URLs have zero probability of collision because the sequencer guarantees unique IDs; custom URLs can collide with a probability of n / (total number of combinations).
- **[CORRECT]** System-generated short URLs have zero probability of collision because the sequencer guarantees unique IDs; custom URLs can collide with a probability of n / (total number of combinations).

  Explanation: The sequencer ensures unique IDs for system-generated short URLs, eliminating collisions. However, user-provided custom URLs can conflict with existing ones. The probability of such a collision is given by:
  $P(\text{collision}) = \frac{n}{\text{Total number of combinations of URLs}}$
  where `n` is the number of existing short URLs. This probability increases as the database grows.
- Both types of short URLs have zero collision probability since all are stored in a globally consistent database.

Question 4: Our system must ensure that no duplicate short URLs are generated for the same long URL. How does it achieve this?

- It always generates a new short URL, even if the long URL already exists, and stores both versions.
- **[CORRECT]** It first checks the cache, then the database for the long URL. If found, it returns the existing short URL; otherwise, it asks the Short URL Generator (SUG) to create a new one and stores it.

  Explanation: The system prevents duplicates by checking the cache and then the database for an existing mapping. If a match exists, the stored short URL is returned. If not, the Short URL Generator creates a new one, which is then stored and returned to the user.
- It relies only on the cache to detect duplicates and skips database verification.
- It generates multiple short URLs for each long URL and lets the user choose one.

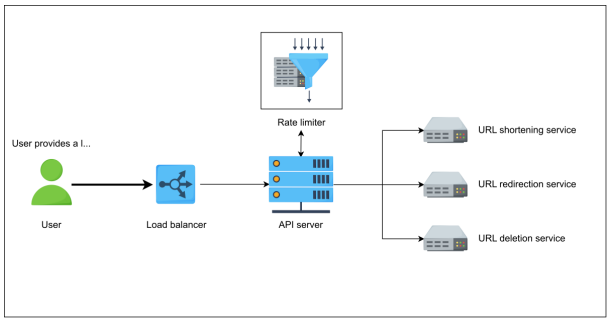
Question 5: When two users send concurrent requests to shorten URLs, how does our system ensure that one request doesn't overwrite the other?

- It allows both writes to happen simultaneously and later merges the results.
- It uses distributed locks across all replicas to manage concurrent writes.
- **[CORRECT]** MongoDB handles this by routing all write operations through a single leader, which serializes requests and prevents race conditions.

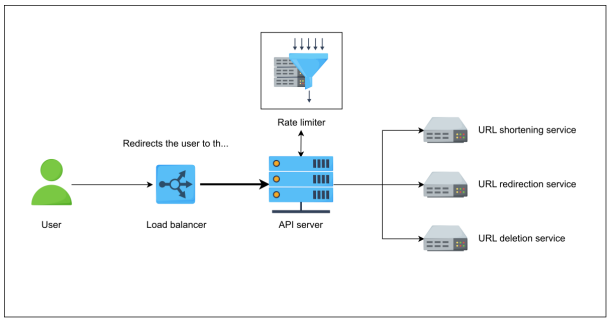
  Explanation: MongoDB maintains consistency through leader-based write serialization and built-in concurrency control. Since all writes go through the leader, simultaneous updates to the same data are automatically ordered, preventing overwrites or race conditions.
- The system queues all incoming requests until the database is idle.

Our system prevents data store bottlenecks by using range-based sequencers and unique ID ranges within short URLs; each request is routed straight to its designated database, ensuring fast lookups and balanced load across all data stores.

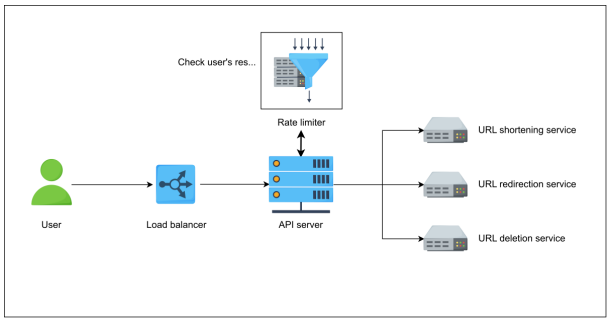
The illustration below depicts how URL shortening, redirection, and deletion work.



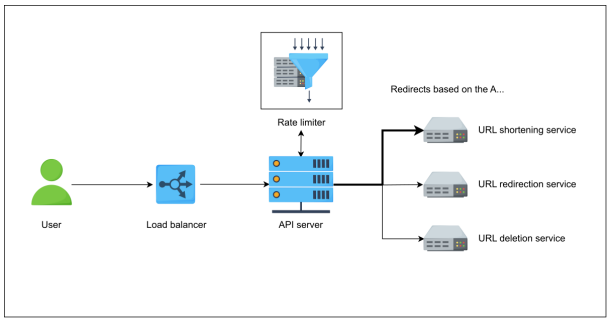
(a) URL shortening: The user initiates the request



(b) URL shortening: The load balancer redirects the request



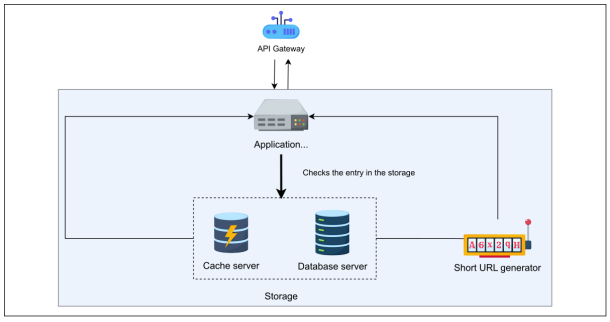
(a) URL shortening: The eligibility of the user's request is checked



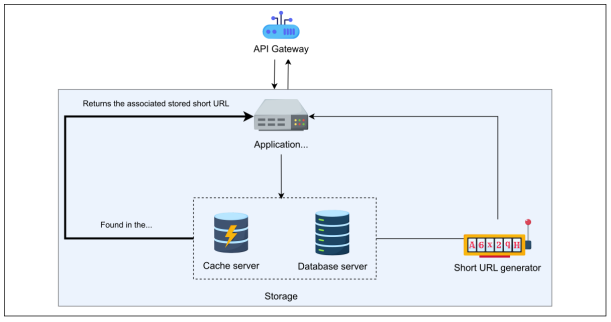
(b) URL shortening: The request is redirected to the concerned service

Consider a situation where your TinyURL service is suddenly getting a massive influx of traffic for URL shortening. The database is starting to slow down under the write load. Propose short-term and long-term solutions to scale the system.

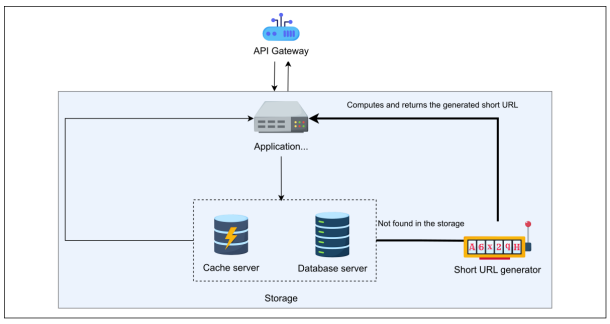
Quiz: Question 1: Upon successful allocation of a custom short URL, how does the system modify its records? **Answer:** Since the custom short URL is the base-58 encoding of an available



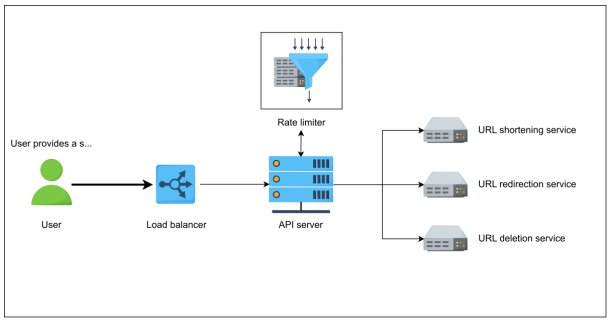
(a) URL shortening: The data store entry checkup is performed



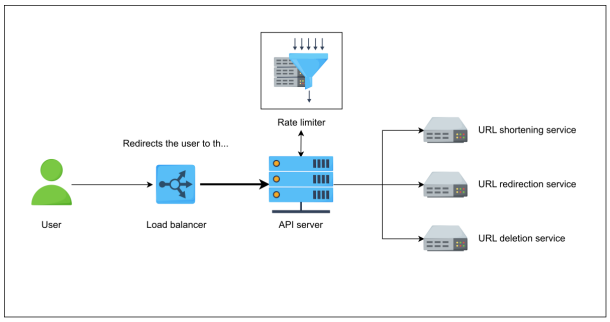
(b) URL shortening: The short URL is found in the data store (Case 1)



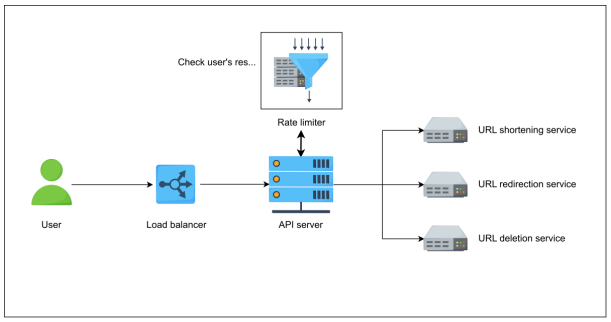
(a) URL shortening: The short URL is computed (Case 2)



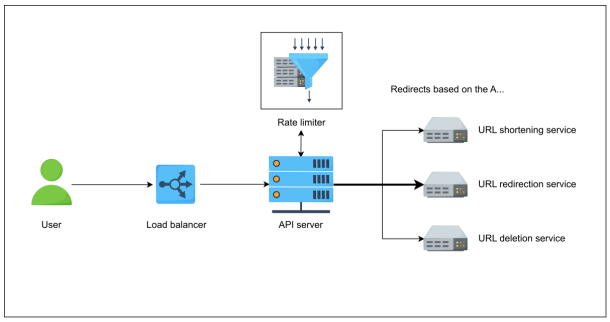
(b) URL redirection: The user clicks on a short URL



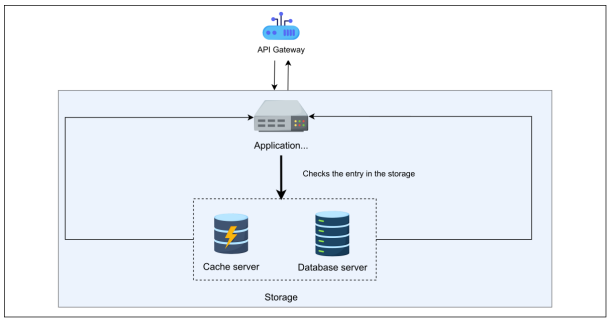
(a) URL redirection: The load balancer redirects the request



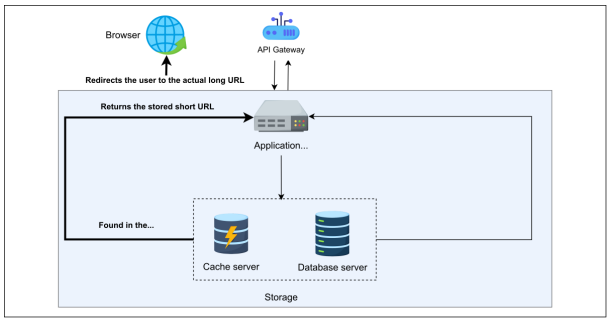
(b) URL redirection: The eligibility of the request is checked



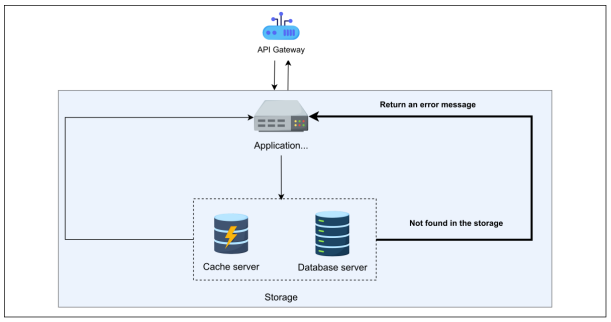
(a) URL redirection: The request is redirected to the concerned service



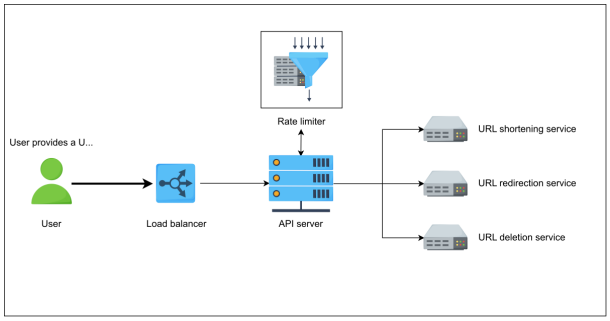
(b) URL redirection: The data store is checked for the entry



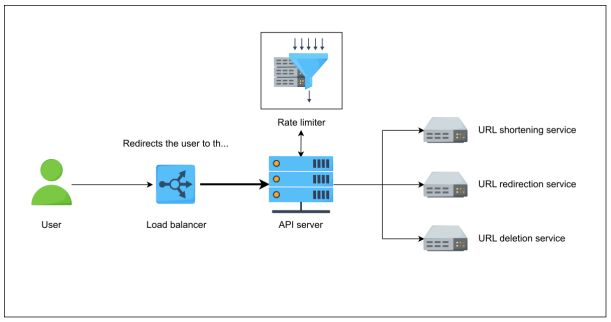
(a) URL redirection: Successful redirection (Case 1)



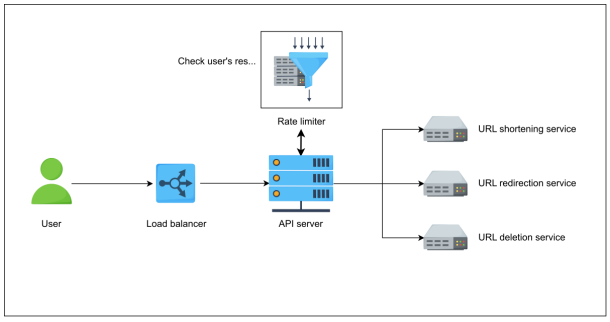
(b) URL redirection: Unsuccessful redirection (Case 2)



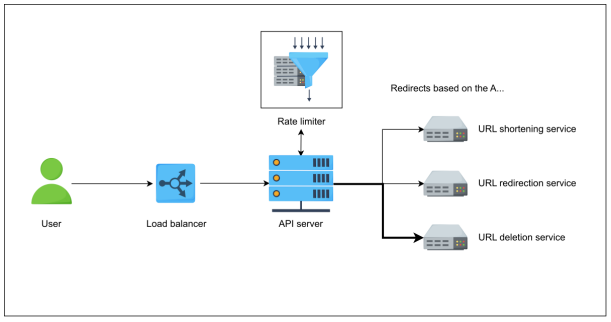
(a) URL deletion: The user initiates the deletion request



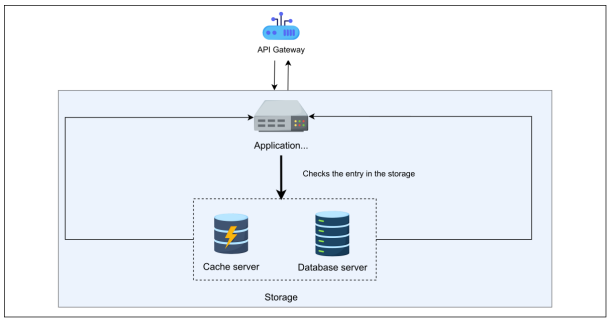
(b) URL deletion: The load balancer redirects the request



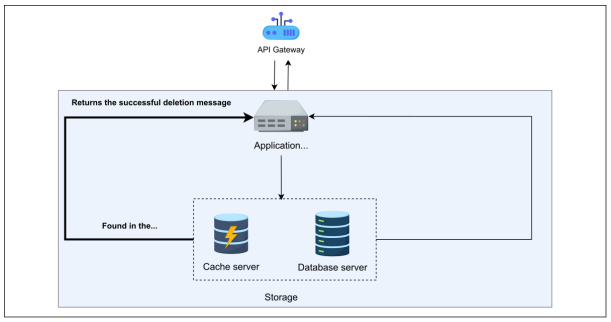
(a) URL deletion: The eligibility of the request is checked



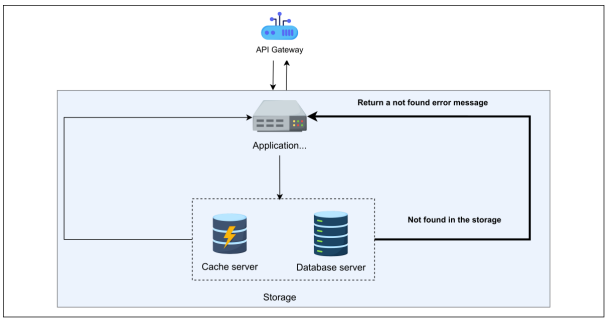
(b) URL deletion: The request is redirected to the concerned service



(a) URL deletion: The data store entry is checked



(b) URL deletion: Successful deletion (Case 1)



(a) URL deletion - unsuccessful deletion (Case 2)

base-10 unique ID, marking that unique ID as unavailable for future use is necessary for the system's integrity. On the backend,

the system accesses the server with the base-10 equivalent unique ID of that specific base-58 short URL. It marks the ID as unavailable in the range, eliminating any chance of reallocating the same ID to any other request. This technique also helps availability. The node generating short URLs will no longer need to maintain a list of used and unused unique IDs in memory. Instead, a database is maintained for each of the lists. For good performance, this database can be NoSQL. The above part explains the post-processing of a custom short URL association. Some further details include the following: 1. Once we generate IDs, we put them in the unused list. As soon as we use an ID from the unused list, we put it in the used list. This eliminates the possibility of duplicate association. 2. As encoding guarantees unique mapping between base-10 and base-58, no two long URLs will have the same short URL.

The following section details how a unique ID is encoded into the final short URL.

1.7 URL Encoding and Readability

We've discussed the overall design of a short URL generator (SUG) in detail, but two aspects need more clarification:

1. How does encoding improve the readability of the short URL?
2. How are the sequencer and the base-58 encoder in the short URL generation related?

Why to use encoding

Our sequencer generates a 64-bit ID in base-10, which can be converted to a base-64 short URL. Base -64 is the most common encoding for alphanumeric strings' generation. However, there are some inherent issues with sticking to the base-64 for this design problem: the generated short URL might have readability issues because of look-alike characters. Characters like `O` (capital o) and `0` (zero), `I` (capital I), and `l` (lower case L) can be confused while characters like `+` and `/` should be avoided because of other system-dependent encodings. So, we slash out the six characters and use base-58 instead of base-64 (includes A-Z, a-z, 0-9, `+` and `/`) for enhanced readability purposes. Let 's look at our base-58 definition.

Base-58

The highlighted cells contain the succeeding characters of the omitted ones: `0`, `O`, `I`, and `l`.

1.8 Converting base-10 to base-58

Since we're converting base-10 numeric IDs to base-58 alphanumeric IDs, explaining the conversion process will be helpful in grasping the underlying mechanism as well as the overall scope of the SUG. To achieve the above functionality, we use the **modulus** function.

Process: We keep dividing the base-10 number by 58, making note of the remainder at each step. We stop

Value	Character	Value	Character	Value	Character	Value	Character
0	1	15	G	30	X	45	n
1	2	16	H	31	Y	46	o
2	3	17	J	32	Z	47	p
3	4	18	K	33	a	48	q
4	5	19	L	34	b	49	r
5	6	20	M	35	c	50	s
6	7	21	N	36	d	51	t
7	8	22	P	37	e	52	u
8	9	23	Q	38	f	53	v
9	A	24	R	39	g	54	w
10	B	25	S	40	h	55	x
11	C	26	T	41	i	56	y
12	D	27	U	42	j	57	z
13	E	28	V	43	k		
14	F	29	W	44	m		

Table 1.1: Base-58

where there is no remainder left. Then we assign the character indexes to the remainders, starting from assigning the recent-most remainder to the left-most place and the oldest remainder to the right-most place.
Example: Let's assume that the selected unique ID is `2468135791013`. The following steps show us the remainder calculations:
Base-10 = 2468135791013

1. 2468135791013 % 58 = 17
2. 42554065362 % 58 = 6
3. 733690782 % 58 = 4
4. 12649841 % 58 = 41
5. 218100 % 58 = 20
6. 3760 % 58 = 48
7. 64 % 58 = 6
8. 1 % 58 = 1

Now, we need to write the remainders in order of the most recent to the oldest order.
Base-58 = [1] [6] [48] [20] [41] [4] [6] [17]
Using the table above, we can write the associated characters with the remainders written above.
Base-58 = 27qMi57J

> **Note:** Both the base-10 numeric IDs and base-64 alphanumeric IDs have 64-bits, as per our sequencer design.

Let's see the example above with the help of the following illustration.

1.9 Converting base-58 to base-10

The decoding process holds equal importance as the encoding process, as we used a decoder in case of custom short URLs generation, as explained in the design lesson.
Process: The process of converting a base-58 number into a base-10 number is also straightforward. We just need to multiply each character index (value column from the table above) by the number of 58s that position holds, and add all the individual multiplication results.
Example: Let's reverse-engineer the example above to see how decoding works.

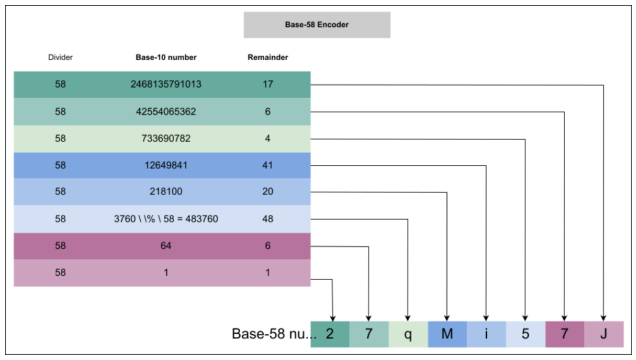


Figure 1.23: How a base-10 number is converted into a base-58 alphanumeric short URL

Base-58: 27qMi57J
$2_{58} = 1 \times 58^7 = 2207984167552$
$7_{58} = 6 \times 58^6 = 228412155264$
$q_{58} = 48 \times 58^5 = 31505124864$
$M_{58} = 20 \times 58^4 = 226329920$
$i_{58} = 41 \times 58^3 = 7999592$
$5_{58} = 4 \times 58^2 = 13456$
$7_{58} = 6 \times 58^1 = 348$
$J_{58} = 17 \times 58^0 = 17$
Base-10 $= 17 + 348 + 13456 + 7999592 + 226329920 + 31505124864 + 228412155264 + 2207984167552$
Base-10 $= 2468135791013$.
This is the same unique ID of base-10 from the previous example.
Let's see the example above with the help of the following illustration.

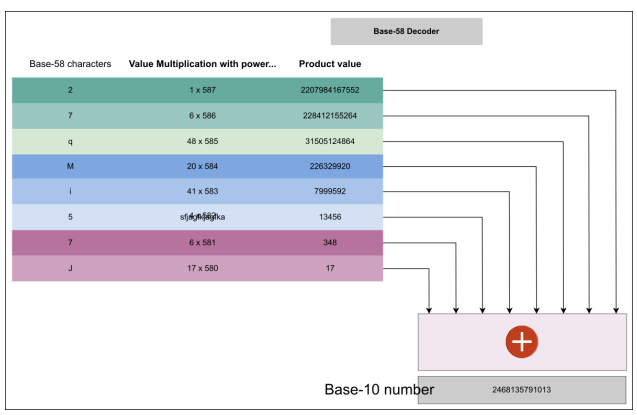


Figure 1.24: How a base-58 number is converted into a base-10 numeric ID

1.10 The scope of the short URL generator

The short URL generator is the backbone of our URL shortening service. The output of this short URL generator depends on the design-imposed limitations, as given below:

- The generated short URL should contain alphanumeric characters.
- None of the characters should look alike.
- The minimum default length of the generated short URL should be six characters.

These limitations define the scope of our short URL generator. We can define the scope, as shown below:

- **Starting range**: Our sequencer can generate a 64-bit binary number that ranges from $1 \rightarrow (2^{64} - 1)$. To meet the requirement for the minimum length of a short URL, we can select the sequencer IDs to start from at least 10 digits, i.e., 1 Billion.
- **Ending point**: The maximum number of digits in sequencer IDs that map into the short URL generator's output depends on the maximum utilization of 64 bits, that is, the largest base-10 number in 64-bits. We can estimate the total number of digits in any base by calculating these two points:
  - The numbers of bits to represent one digit in a base-n. This is given by $\log_2 n$.
  - Number of digits $= \frac{\text{Total bits available}}{\text{Number of bits to represent one digit}}$

Let's see the calculations above for both the base-10 and base-58 mathematically:

- - **Base-10:**
    - The number of bits needed to represent one decimal digit $= \log_2 10 = 3.13$
    - The total number of decimal digits in 64-bit numeric ID $= \frac{64}{3.13} = 20$
  - **Base-58:**
    - The number of bits needed to represent one decimal digit $= \log_2 58 = 5.85$
    - The total number of base-58 digits in a 64-bit numeric ID $= \frac{64}{5.85} = 11$

  Maximum digits: The calculations above show that the maximum digits in the sequencer generated ID will be 20 and consequently, the maximum number of characters in the encoded short URL will be 11.

The sequencer's largest number, $2^{64} - 1$ (that's 18,446,744,073,709,551,615), becomes `jpXCZedGfVQ` when converted to base-58; the longest possible short URL our system can generate!

Our system currently uses 10-digit and beyond sequencer IDs. What if we wanted to make them shorter? How could we use the sequencer ID range below 10 digits more effectively?

Question:

Using shorter sequencer IDs for custom short links

Reference Answer:

We can reserve sequencer IDs below 10 digits for premium users' custom short links. This allows us to:

- Reuse the lower ID range efficiently
- Offer shorter URLs (under six characters) For example, if a user requests `abc`, we assign it to their long URL and mark its decoded base-10 ID (e.g., 113019) as used, ensuring uniqueness.

1.11 The sequencer's lifetime

The number of years that our sequencer can provide us with unique IDs depends on two factors:

- - Total numbers available in the sequencer $= 2^{64} - 10^9$ (starting from 1 Billion as discussed above)
  - Number of requests per year = 200 Million per month $\times$ 12 = 2.4 Billion (as assumed in *Requirements*)

So, taking the above two factors into consideration, we can calculate the expected life of our sequencer.
The lifetime of the sequencer $= \frac{\text{total numbers available}}{\text{yearly requests}} = \frac{2^{64} - 10^9}{2.4 \text{ Billion}} = 7,686,143,363.63$ years

Therefore, our service can run for a long time before the range depletes.

After detailing the complete architecture, from high-level components down to the specifics of encoding, let's validate our design against the requirements we established at the start.

1.12 Requirements compliance

The last stage of a System Design is to evaluate it as per the non-functional requirements mentioned initially. Let's look at each metric one by one.

Availability

We need high availability for users generating new short URLs and redirecting them based on the existing short URLs.
Most of our building blocks, like databases, caches, and application servers have built-in replication that ensures availability and fault tolerance. The short URL generation system will not impact the availability either, as it depends on an easily replicable database of available and used unique IDs.
To handle disasters, we can perform frequent backups of the storage and application servers, preferably twice a day, as we can't risk losing URLs data. We can use the Amazon S3 storage service for backups, as it facilitates cross-zonal replicating and restoration as well. In the worst-case scenario, we might lose 3.3 million (with 6.6 million daily requests assumed) newly generated short URLs that are not backed up on that specific day.
Our design uses **global server load balancing (GSLB)** to handle our system traffic. It ensures intelligent request distribution among different global servers, especially in the case of on-site failures.
We also apply a limit on the requests from clients to secure the intrinsic points of failures. To protect the system against DoS attacks, we use rate limiters between the client and web servers to limit each user's resource allocation. This will ensure a good and smooth traffic influx and mitigate the exploitation of system resources.

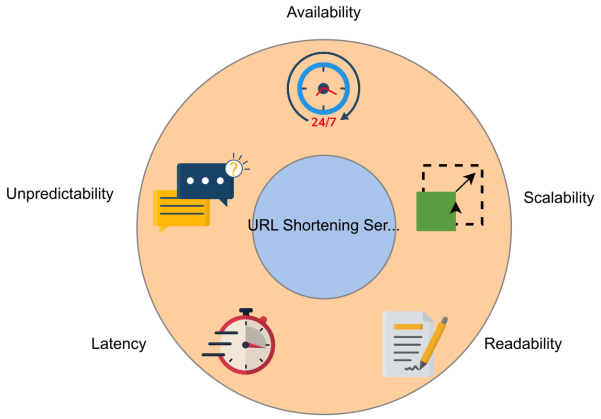


Figure 1.25: Guarantees provided by our URL shortening service

Scalability

Our design is scalable because our data can easily be distributed among horizontally sharded databases. We can employ a consistent hashing scheme to balance the load between the application and database layers.

We can also use the range-based sharding while scaling our database server. However, if we partition our database based on a predefined range, it might lead to imbalanced partitions due to the variable distribution of URLs in each range. An example of this could be the distribution based on the first letter of the URL.

Our choice of the database for mapping URLs, MongoDB, also facilitates horizontal scaling. Some interesting reasons for selecting a NoSQL database are:

1. When a user accesses our system without logging in, our system doesn't save the `UserID`. Since we're flexible with storing data values, and it aligns more with the schematic flexibility provided by the NoSQL databases, using one for our design is preferable.

2. Scaling a traditional relational database horizontally is a daunting process and poses challenges to meeting our scalability requirements. We want to scale and automatically distribute our system's data across multiple servers. For this requirement, a NoSQL database would best serve our purpose.

Moreover, the large number of unique IDs available in the sequencer's design also ensures the scalability of our system.

Readability

The use of a base-58 encoder, instead of the base-64 encoder, enhances the readability of our system. We divide the readability into two sections:

- **Distinguishable characters** like `0` (zero), `O` (capital o), `I` (capital i), and `l` (lower case L) are eliminated, excluding the possibility of mistaking a character for another look-alike character.
- **Non-alphanumeric characters** like `+` (plus) and `/` (slash) are also eliminated to only have alphanumeric characters in short URLs. Second, it also helps avoid other system-dependent encodings and makes the URLs readily available for all modern file systems and URLs schemes. Such characters may lead to undesired behavior and output during parsing.

This non-functional requirement enhances the user interactivity of our system and makes short URL usage less error-prone for the users.

Latency

Our system ensures low latency with its following features:

- Even the most time-consuming step across the short URL generation process, encoding, takes a few milliseconds. The overall time to generate a short URL is relatively low, ensuring there are no significant delays in this process.
- Our system is redirection-heavy. Writing on the database is minimal compared to reading, and its performance depends on how well it copes with all the redirection requests, compared to the shortening requests. We deliberately chose MongoDB because of its low latency and high throughput in reading-intensive tasks.
- Moreover, the probability of the user using the freshly generated short URL in the next few seconds is relatively low. During this time, synchronous replication to other locations is feasible and therefore adds to the overall low latency of the system for the user.
- The deployment of a distributed cache in our design also ensures that the system redirects the user with the minimum possible delay.

As a result of such design modifications, the system enjoys low latency and high throughput, providing good performance.

Unpredictability

One of the requirements is to make our system's short URLs unpredictable, enhancing the security of our system.
As the sequencer generates unique IDs in a sequence and distributes ranges among servers. Each server has a pre-assigned range of unique IDs, assigning them serially to the requests will make it easy to predict the following short URL. To counter it, we can randomly select a unique ID from the available ones and associate it to the long URL, encompassing the unpredictability of our system.
The following table summarizes the techniques for achieving non-functional requirements in a TinyURL system:

Non-functional Requirements Compliance

Now it's time to assess your understanding with the following quiz:

Requirements	Techniques
Availability	• The Amazon S3 service backs up the storage and cache servers daily. We can restore them upon fault occurrence. • Global server load balancing to handle the system's traffic. • Rate limiters to limit each user's resource allocation.
Scalability	• Horizontal sharding of the database. • Distribution of the data based on consistent hashing. • MongoDB - as the NoSQL database.
Readability	• Introduction of the base-58 encoder to generate short URLs. • Removal of non-alphanumeric characters. • Removal of look-alike characters.
Latency	• Unnoticeable delay in the overall operation. • MongoDB, because of its low latency and high throughput in reading tasks. • Distributed cache to minimize the service delays.
Unpredictability	• Randomly selecting and associating an ID to each request, from the pool of unused and readily available unique IDs.

Table 1.2: Non-functional Requirements Compliance

Quiz

Question 1: In the URL shortener system's design, the main reason to change the base of the sequencer to `58` was:

- To make the generated short URLs more secure.
  Explanation: Security is independent of the base of the sequencer.
- **[CORRECT]** To enhance the readability of the generated short URLs.
  Explanation: We change the base to 58 to remove the look-alike characters.
- To improve the system's availability.
  Explanation: Availability has no direct correspondence with the base of the sequencer.
- To ensure the system's low latency.
  Explanation: Low-latency has no direct correspondence with the base of the sequencer.

Question 2: In the system design phase, which reason below was the reason why we decided to use MongoDB?
(Select all that apply.)

- It has a higher write throughput as compared to reading.
  Explanation: The opposite is true for MongoDB.
- **[CORRECT]** It ensures atomicity upon concurrent writes.
- It is the only NoSQL database for more specific problems like TinyURL.
  Explanation: It is a very vague statement and, more importantly, false. Every database has its pros and cons.
- It's easier to scale MongoDB as compared to other leader-less NoSQL databases.
  Explanation: All NoSQL databases are easily scalable, and such a comparison is not readily available.

Question 3: (Select all that apply.) To meet the functional requirement of creating a custom alias for the short link, which of the following restriction(s) do we need to impose on the custom_alias variable for the API call?
You can select multiple correct answers.
(Select all that apply.)

- **[CORRECT]** The length of the custom_alias shouldn't be greater than the allowed character limit of the short links.
  Explanation: In our case, we have limited to 11 alpha-numeric characters from base-58.
- The user must declare the value of the custom_alias variable.
  Explanation: It's not a requirement of the system we designed, as the default value is `None`.
- Every time a user creates a short link, a single user must enter the same custom_alias.
  Explanation: It's a possibility but not a requirement; we can update the long URL against the custom URLs, but it's not mandatory for a user.

- **[CORRECT]** The custom_alias should not contain the look-alike removed characters.

  Explanation: It's an essential requirement to just include the allowed base-58 characters for custom_alias.

Question 4: Which strategy did we adopt to counter the predictability of the sequencer-generated unique IDs?

- We changed the base of the sequencer to `58`.
- **[CORRECT]** We randomly selected a unique ID from the available pool for each short URL generation.
- We added a load balancer before the sequencer.
- We eliminated the alpha-numeric characters.

Having designed the system and confirmed that it meets all our critical non-functional goals, we can now summarize our approach.

1.13 Conclusion

The URL shortening system is an effective service with multiple advantages. Our design of the URL shortening service is simple, yet it fulfills all the requirements of a performant design. The key features offered by our design are:

1. A dynamic short URL range
2. Improved readability

A possible addition could be the introduction of (local) salt to further increase the unpredictability (security) of the design.